

Day 25: プロフィール編集を実装しよう

今日のゴール

ユーザーが自分のプロフィール情報（名前、メールアドレス、パスワード）を編集できる機能を実装します。

【スクリーンショット: プロフィール編集画面】

なぜこれを作るのか？

ユーザー自身が情報を管理できる機能です。プロフィール編集は名刺の更新のようなもの。引っ越しや転職で連絡先が変わったら、名刺を新しく作り直します。同様に、メールアドレスが変わったり、名前の表記を変えたいときに、自分で更新できると便利です。

実装ステップ一覧

ステップ	作業内容	所要時間
Step 1	プロフィールページ作成	10分
Step 2	基本情報編集フォーム	15分
Step 3	パスワード変更フォーム	15分
Step 4	更新API呼び出し	10分

合計時間: 約50分

Step 1: プロフィールページ作成（10分）

実装:

```
// filepath: src/app/profile/page.tsx
'use client';

import { useSession } from 'next-auth/react';
import { Box, Typography, CircularProgress } from '@mui/material';

export default function ProfilePage() {
  const { data: session, status } = useSession();

  if (status === 'loading') {
    return (
      <Box sx={{ display: 'flex', justifyContent: 'center' }}>
        <CircularProgress />
      </Box>
    );
  }

  return (
    <Box sx={{ p: 3 }}>
      <Typography variant="h4">プロフィール編集</Typography>
    </Box>
  );
}
```

確認ポイント: /profileにアクセスして「プロフィール編集」が表示される

Step 2: 基本情報編集フォーム（15分）

実装:

```
// filepath: src/app/profile/page.tsx
import { useState, useEffect } from 'react';
import {
  TextField,
  Button,
  Paper,
  Divider,
} from '@mui/material';
import { api } from '@trpc/react';

export default function ProfilePage() {
  const { data: session } = useSession();
  const { data: user } = api.user.getMe.useQuery();

  const [name, setName] = useState('');
  const [email, setEmail] = useState('');

  useEffect(() => {
    if (user) {
      setName(user.name || '');
      setEmail(user.email);
    }
  }, [user]);

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    console.log({ name, email });
  };

  return (
    <Box sx={{ p: 3, maxWidth: 600 }}>
      <Typography variant="h4" sx={{ mb: 3 }}>プロフィール編集</Typography>

      <Paper sx={{ p: 3, mb: 3 }}>
        <Typography variant="h6" sx={{ mb: 2 }}>
          基本情報
        </Typography>
      </Paper>
    </Box>
  );
}
```

```

</Typography>
<form onSubmit={handleSubmit}>
  <TextField
    fullWidth
    label="名前"
    value={name}
    onChange={(e) => setName(e.target.value)}
    sx={{ mb: 2 }}
  />
  <TextField
    fullWidth
    label="メールアドレス"
    type="email"
    value={email}
    onChange={(e) => setEmail(e.target.value)}
    sx={{ mb: 2 }}
  />
  <Button type="submit" variant="contained">
    更新
  </Button>
</form>
</Paper>
</Box>
);
}

```

確認ポイント: 現在の名前とメールアドレスが表示される

Step 3: パスワード変更フォーム（15分）

実装:

```
// filepath: src/app/profile/page.tsx
export default function ProfilePage() {
  const [currentPassword, setCurrentPassword] = useState('');
  const [newPassword, setNewPassword] = useState('');
  const [confirmPassword, setConfirmPassword] = useState('');
  const [passwordError, setPasswordError] = useState('');

  const handlePasswordSubmit = (e: React.FormEvent) => {
    e.preventDefault();

    if (newPassword !== confirmPassword) {
      setPasswordError('パスワードが一致しません');
      return;
    }

    if (newPassword.length < 8) {
      setPasswordError('パスワードは8文字以上にしてください');
      return;
    }

    setPasswordError('');
    console.log('パスワード変更');
  };

  return (
    <Box sx={{ p: 3, maxWidth: 600 }}>
      {/* 基本情報フォーム */}

      <Paper sx={{ p: 3 }}>
        <Typography variant="h6" sx={{ mb: 2 }}>
          パスワード変更
        </Typography>
        <form onSubmit={handlePasswordSubmit}>
          <TextField
            fullWidth
            type="password"
          />
        </form>
      </Paper>
    </Box>
  );
}
```

```

        label="現在のパスワード"
        value={currentPassword}
        onChange={(e) => setCurrentPassword(e.target.
        sx={{ mb: 2 }}
    />
    <TextField
        fullWidth
        type="password"
        label="新しいパスワード"
        value={newPassword}
        onChange={(e) => setNewPassword(e.target.valu
        sx={{ mb: 2 }}
    />
    <TextField
        fullWidth
        type="password"
        label="新しいパスワード (確認) "
        value={confirmPassword}
        onChange={(e) => setConfirmPassword(e.target.
        error={Boolean(passwordError)}
        helperText={passwordError}
        sx={{ mb: 2 }}
    />
    <Button type="submit" variant="contained">
        パスワードを変更
    </Button>
</form>
</Paper>
</Box>
);
}

```

確認ポイント: パスワード変更フォームが表示される

Step 4: 更新API呼び出し (10分)

実装:


```
// filepath: src/app/profile/page.tsx
export default function ProfilePage() {
  const utils = api.useUtils();

  const updateProfileMutation = api.user.updateProfile.useMutation({
    onSuccess: () => {
      alert('プロフィールを更新しました');
      utils.user.getMe.invalidate();
    },
  });

  const updatePasswordMutation = api.user.updatePassword.useMutation({
    onSuccess: () => {
      alert('パスワードを変更しました');
      setCurrentPassword('');
      setNewPassword('');
      setConfirmPassword('');
    },
  });

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    updateProfileMutation.mutate({ name, email });
  };

  const handlePasswordSubmit = (e: React.FormEvent) => {
    e.preventDefault();

    if (newPassword !== confirmPassword) {
      setPasswordError('パスワードが一致しません');
      return;
    }

    if (newPassword.length < 8) {
      setPasswordError('パスワードは8文字以上にしてください');
      return;
    }
  };
}
```

```

    }

    setPasswordError('');
    updatePasswordMutation.mutate({
      currentPassword,
      newPassword,
    });
  };

  return (
    // UI は同じ
    <Button
      type="submit"
      variant="contained"
      disabled={updateProfileMutation.isPending}
    >
      {updateProfileMutation.isPending ? '更新中...' : '更新'}
    </Button>
  );
}

```

確認ポイント: プロフィールとパスワードが更新される

学んだこと

- **複数のフォーム:** 1ページ内で複数のsubmit処理を分離
- **パスワードバリデーション:** 一致チェックと長さチェック
- **Paper で区切り:** 視覚的にセクションを分離
- **error と helperText:** フォームのエラー表示
- **alert():** 簡易的な成功メッセージ表示

今日のまとめ

- ☐ プロフィールページを作成できた
- ☐ 基本情報編集フォームを実装できた
- ☐ パスワード変更フォームを実装できた
- ☐ 更新APIを呼び出した

⚠ つまづきポイント

問題	原因	解決策
パスワードが平文で送信される	バリデーションだけでAPI呼び出し	バックエンドでハッシュ化を確認
確認パスワードが一致しない	判定が甘い	<code>!==</code> で厳密比較
フォームが空にならない	submit 後に state をクリアしていない	onSuccess で <code>setXxx("")</code> を実行

次回予告

Day 26では、エラー対策とデバッグ方法を学びます。